Lab 2 — Greedy Regret Heuristics for Selective TSP with Node Costs

1. Description of the problem

Same problem as in Lab 1 (`data/TSPA.csv`, `data/TSPB.csv`, `n = 200` nodes, select exactly `k = ceil(n/2) = 100` nodes, Hamiltonian cycle minimizing total edge length plus selected node costs; distances are Euclidean rounded to integers, methods use only the distance matrix and node costs). Lab 2 adds regret-based constructive heuristics on top of the Lab 1 baselines (random, NN at end, NN anywhere, greedy cycle).

2. Implemented algorithms — pseudocode

Common notation: `n` nodes, select `k`, `D` distance matrix, `C` node costs, `used[]` visited flag. Lab 1 baselines (`random_tour`, `nearest_neighbor_at_end`, `nearest_neighbor_anywhere`, `greedy_cycle`) are reused unchanged.

2.1 Insertion deltas

Path (extended NN) insertion of candidate `c` into path `T = (t0, …, t{m-1})`: before first `D[c,t0]`, after last `D[t{m-1},c]`, inside edge `(tj,t{j+1})`: `D[tj,c] + D[c,t{j+1}] − D[tj,t{j+1}]`, each plus `C[c]`. Cycle insertion on edge `(tj, t{(j+1) mod m})`: `D[tj,c] + D[c,t{(j+1) mod m}] − D[tj,t{(j+1) mod m}] + C[c]` (for `|T| = 1`: `2·D[t0,c] + C[c]`).

2.2 Greedy 2-regret (`greedy_2regret_nn`, `greedy_2regret_cycle`)

```
REGRET(s, D, C, k):
   tour <- [s]; used[s] <- true
   repeat k-1 times:
       for each c not used:
           deltas <- insertion cost of c over all feasible positions (+C[c])
           regret(c) <- 2nd_best(deltas) - best(deltas)   # 0 if a single position
       c* <- argmax regret (tie-break: smaller best delta)
       insert c* at its best position; used[c*] <- true
   return tour
```

2.3 Weighted sum: 2-regret + best change (`greedy_weighted_nn`, `greedy_weighted_cycle`)

Same loop, but the candidate maximizes:

```
score(c) = w_regret * regret(c) - w_cost * best_delta(c)
```

Default `w_regret = w_cost = 1.0` (also tested `(1.0, 0.5)` and `(0.5, 1.0)`). Pure best-change greedy is the special case `w_regret = 0`.

3. Computational experiment

Each constructive method is run from every start node (200 runs); random: 200 solutions. Weighted-sum methods use equal weights in the main comparison; other weight settings are reported separately in 3.2.

3.1 Results: min / max / mean (Lab 1 + Lab 2 methods)

instance	method	min	max	mean
TSPA	random	237157	289146	263626.5
TSPA	nearest_neighbor_at_end	84362	90075	87476.6
TSPA	nearest_neighbor_anywhere	72328	77022	74347.0

instance	method	min	max	mean
TSPA	greedy_cycle	72590	**75281**	73793.4
TSPA	greedy_2regret_nn	110950	135771	121360.3
TSPA	greedy_2regret_cycle	112806	128089	120378.8
TSPA	greedy_weighted_nn	72419	78504	75549.4
TSPA	greedy_weighted_cycle	**72011**	75532	**73353.5**
TSPB	random	188874	232618	207052.8
TSPB	nearest_neighbor_at_end	56007	63166	59397.3
TSPB	nearest_neighbor_anywhere	52915	66164	59480.3
TSPB	greedy_cycle	55655	63734	59649.9
TSPB	greedy_2regret_nn	68761	79756	73649.8
TSPB	greedy_2regret_cycle	68150	77274	72484.8
TSPB	greedy_weighted_nn	**50485**	**61406**	**55698.9**
TSPB	greedy_weighted_cycle	52731	63370	57863.3

3.2 Weight sensitivity (weighted-sum methods)

Equal weights `(1.0, 1.0)` are the default used in 3.1.

instance	base	w_regret	w_cost	min	max	mean
TSPA	nn	1.0	1.0	72419	78504	75549.4
TSPA	nn	1.0	0.5	73614	79873	76408.1
TSPA	nn	0.5	1.0	72134	78210	75279.7
TSPA	cycle	1.0	1.0	72011	75532	73353.5
TSPA	cycle	1.0	0.5	72575	76095	74277.2
TSPA	cycle	0.5	1.0	72049	75274	73189.0
TSPB	nn	1.0	1.0	50485	61406	55698.9
TSPB	nn	1.0	0.5	51286	60794	54710.1
TSPB	nn	0.5	1.0	52302	63489	56150.8
TSPB	cycle	1.0	1.0	52731	63370	57863.3
TSPB	cycle	1.0	0.5	52788	63809	59659.1
TSPB	cycle	0.5	1.0	52099	62556	55714.3

Reading:

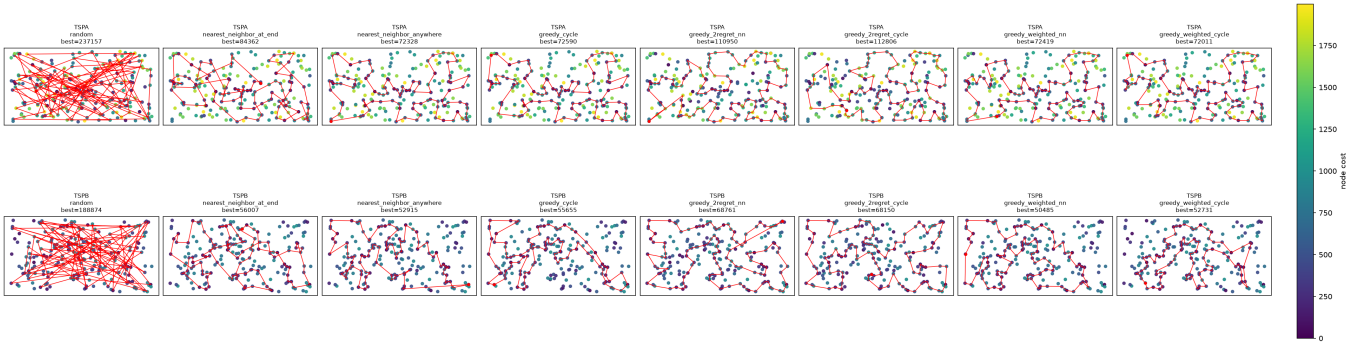
- Pure 2-regret is by far the weakest constructive method here (TSPA best 110950/112806 vs NN-END 84362; TSPB 68761/68150 vs 56007). Maximizing regret alone keeps picking 'urgent' nodes with large absolute insertion cost — urgency without price awareness fails on this cost-augmented objective.
- Adding the best-change term (equal weights) fixes it: weighted variants beat every Lab 1 baseline on best tour (TSPA 72011 vs 72328; TSPB 50485 vs 52915) and take the best mean on both instances (73793.4 → 73353.5 on TSPA; 59649.9/59480.3 → 55698.9 on TSPB).
- Cycle vs path base stays instance-dependent: on TSPA the cycle-based weighted method wins best and mean; on TSPB the path-based one wins best and mean, while the cycle keeps a better worst case.

- Weight sensitivity is mild: on TSPA all six configs land within ~2% best (72011–73614); on TSPB the regret-heavy `(1.0, 0.5)` path config gives the best mean (54710.1) at a slightly worse best (51286 vs 50485). Equal weights are a solid default, not a knife-edge optimum.

3.3 Visualization of best solutions

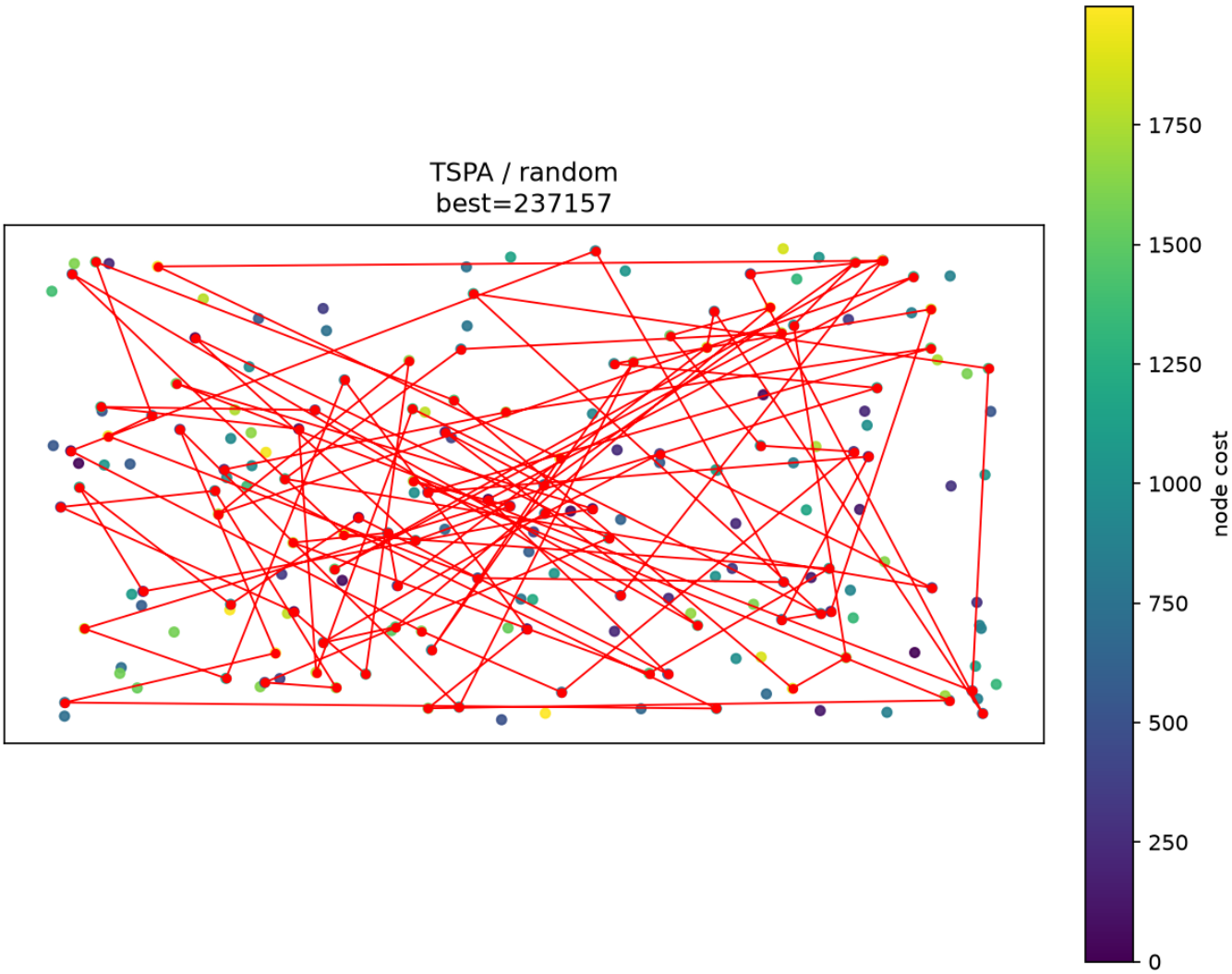
Same encoding as Lab 1: background dots = all 200 nodes, color (`viridis`) = node cost; red polyline + dots = best Hamiltonian cycle (closed, `k=100` nodes).
Overview grid (2 instances × 8 methods):

Best tours grid

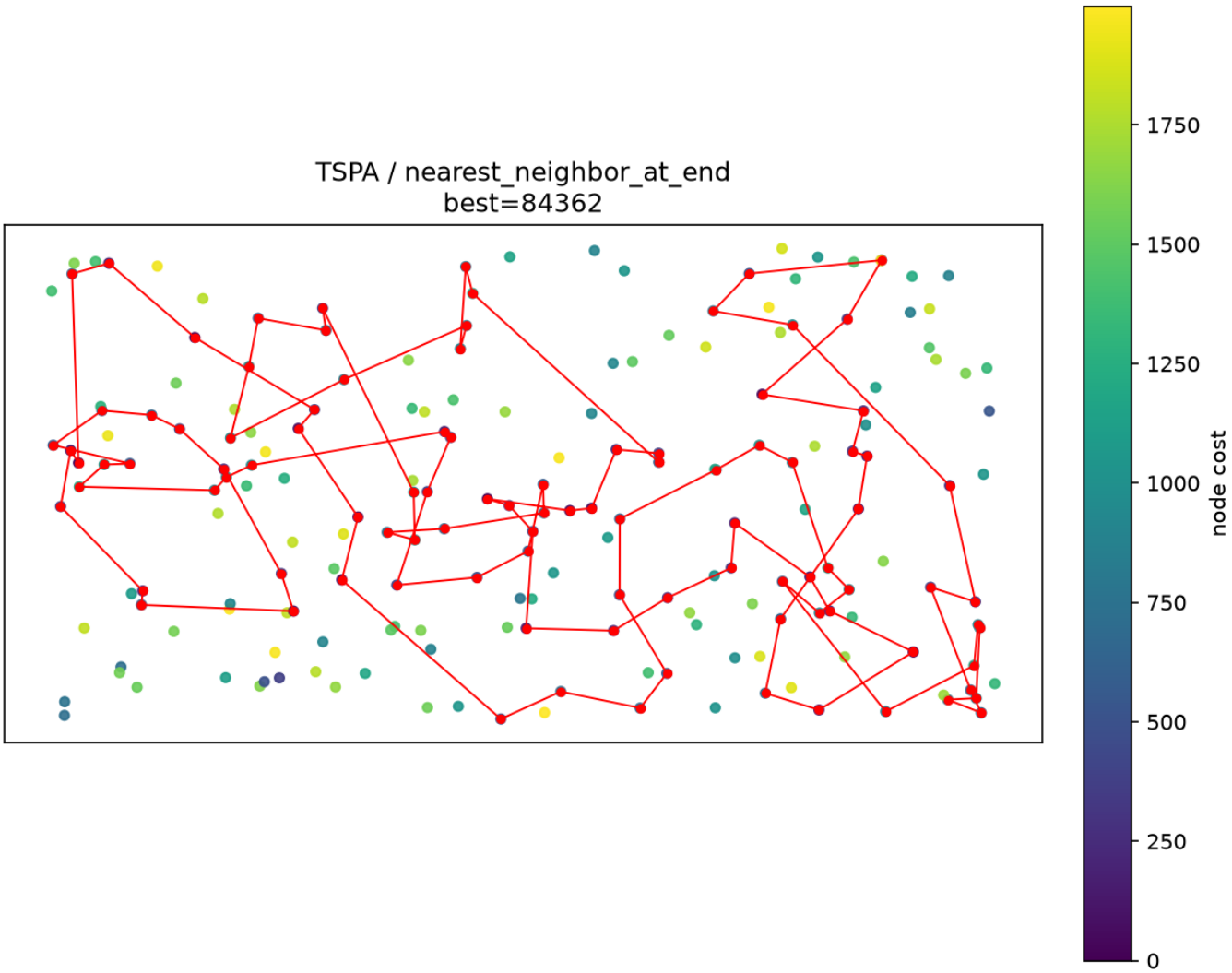


Individual best tours:

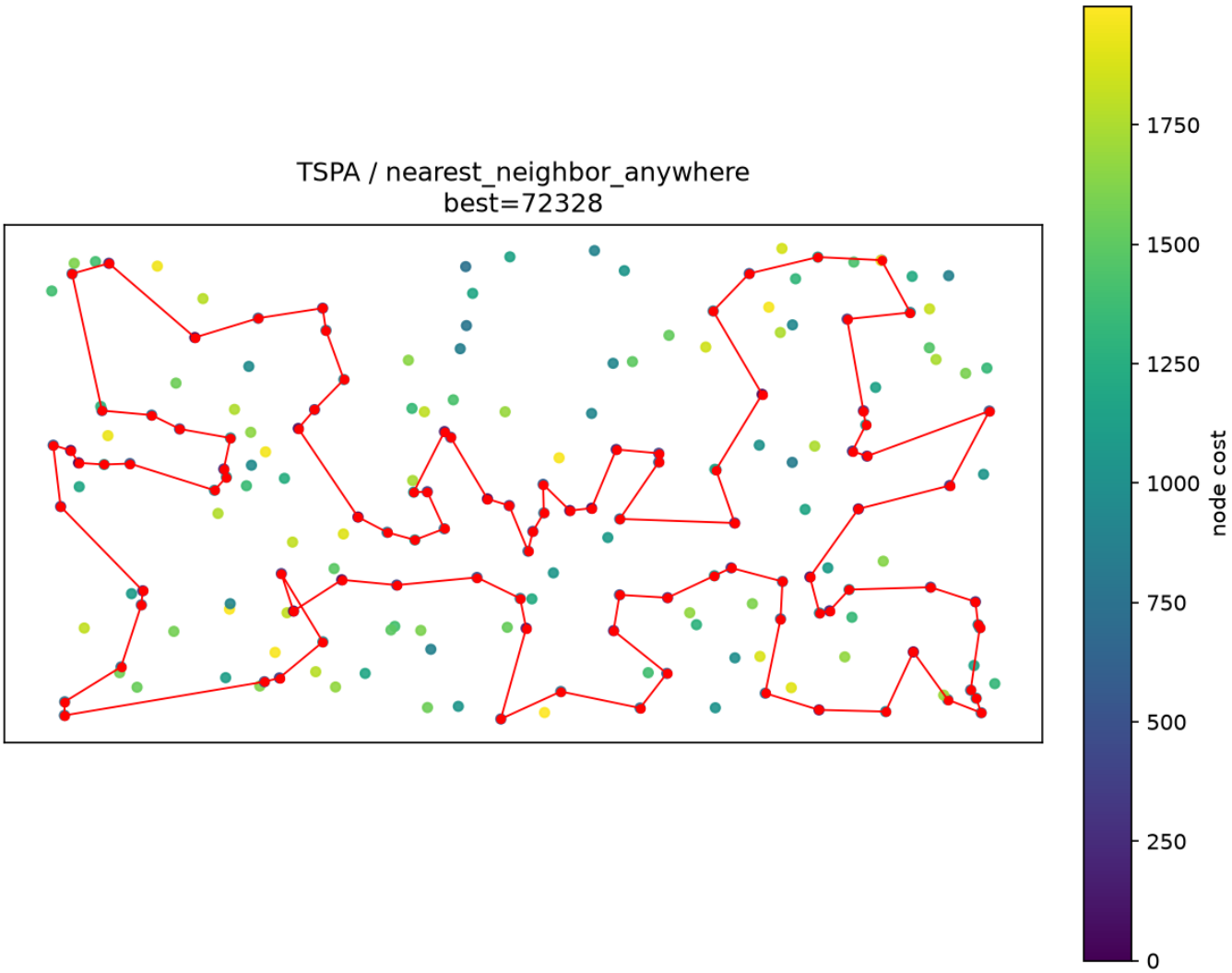
TSPA / random — best=237157



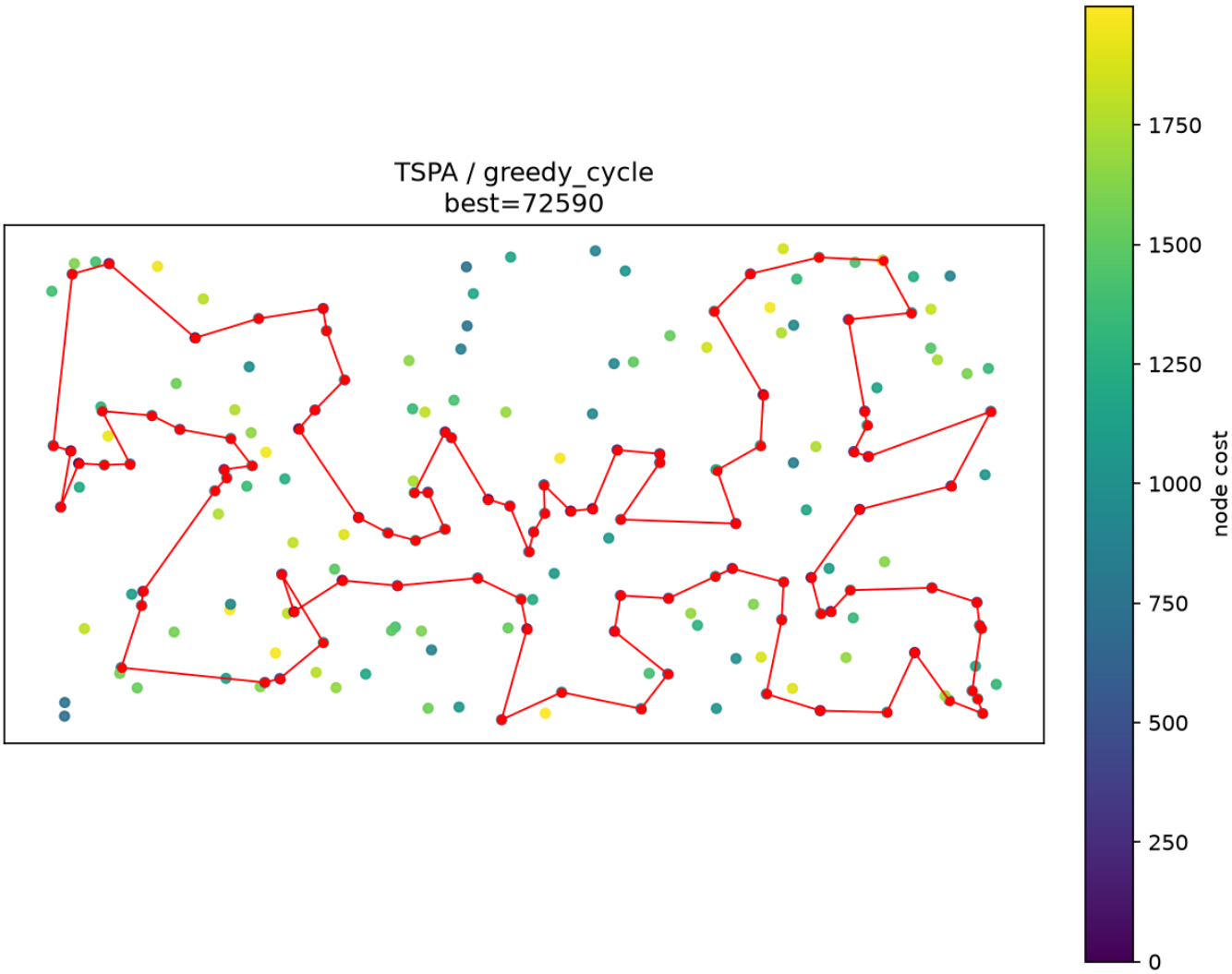
TSPA / nearest_neighbor_at_end — best=84362



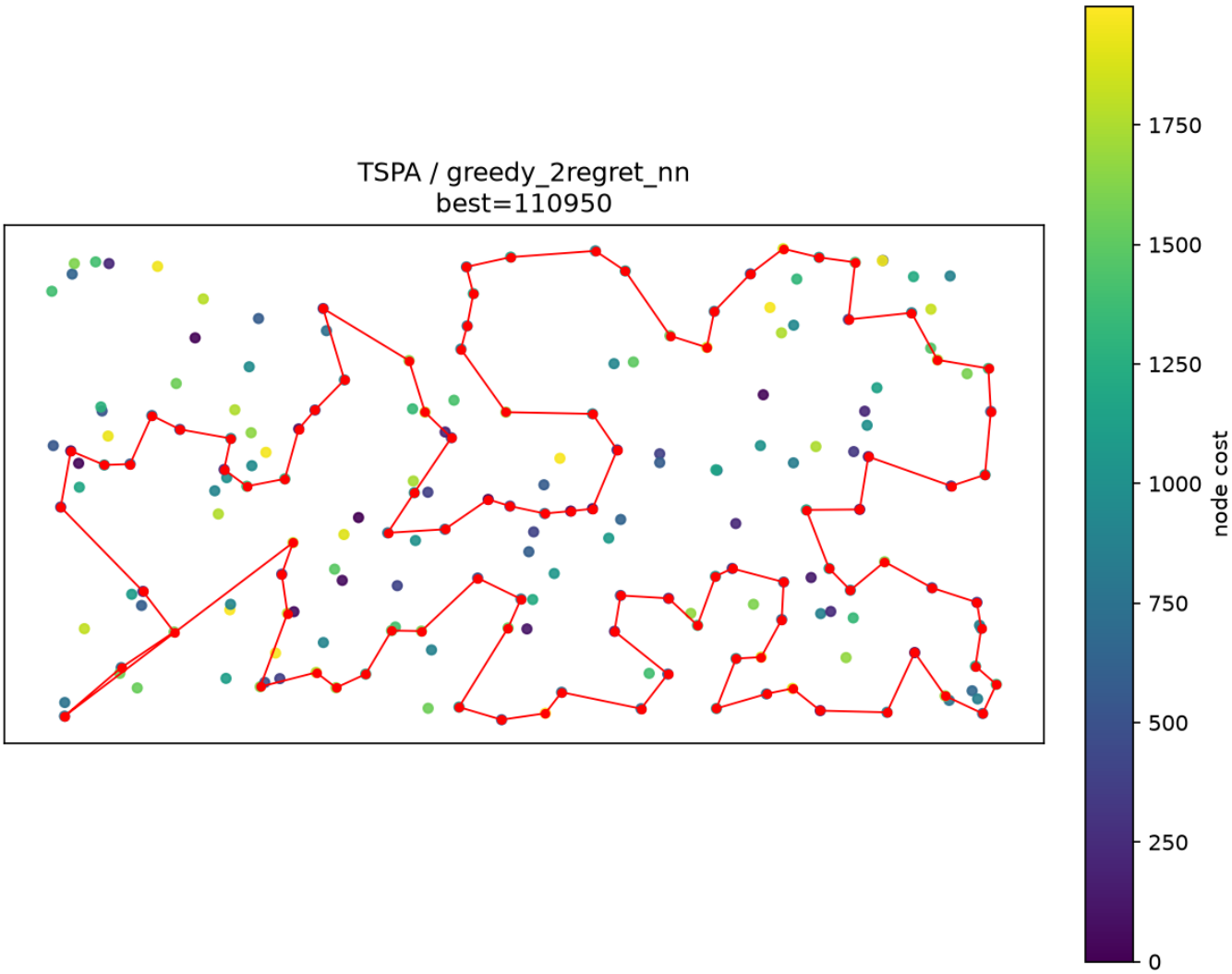
TSPA / nearest_neighbor_anywhere — best=72328



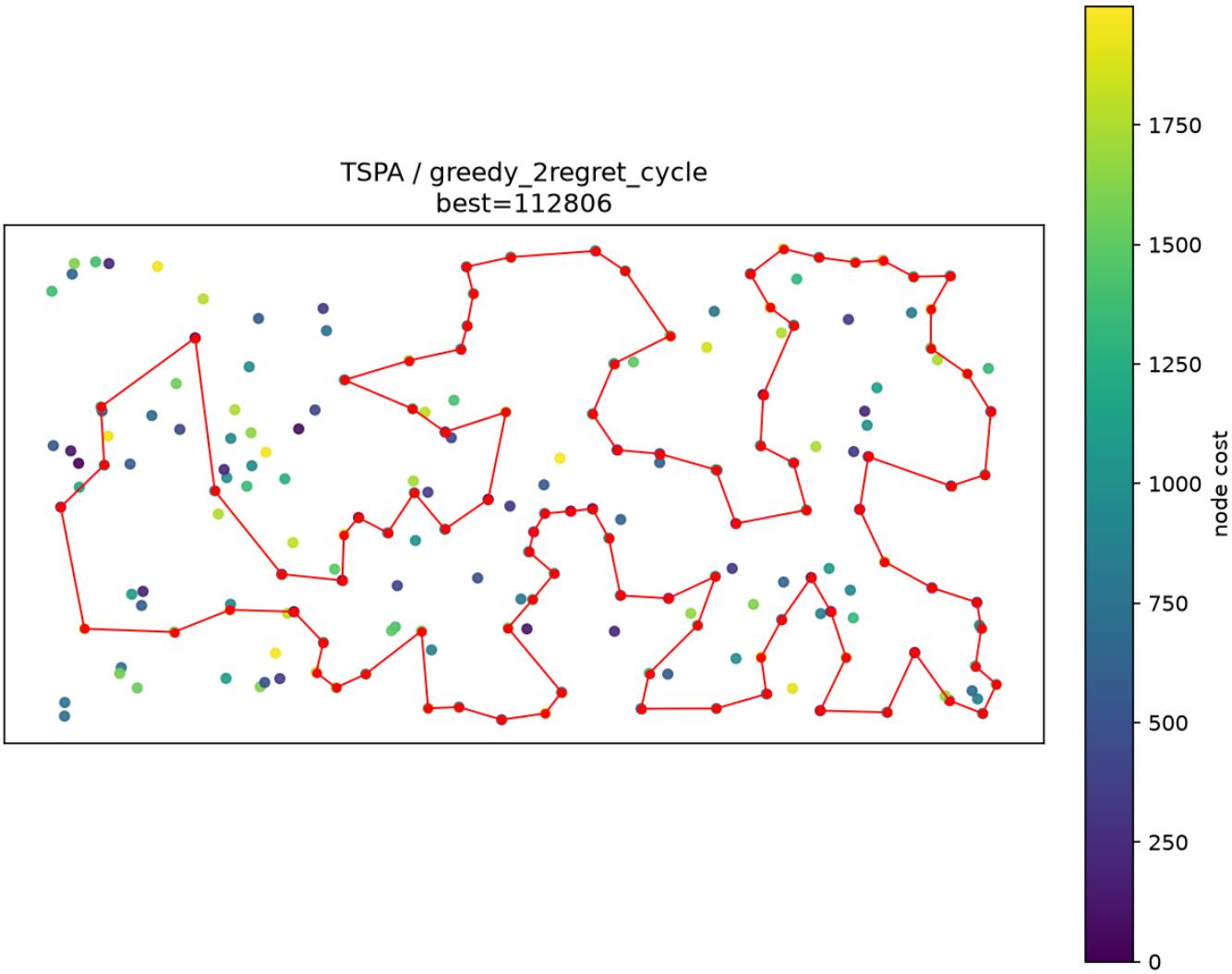
TSPA / greedy_cycle — best=72590



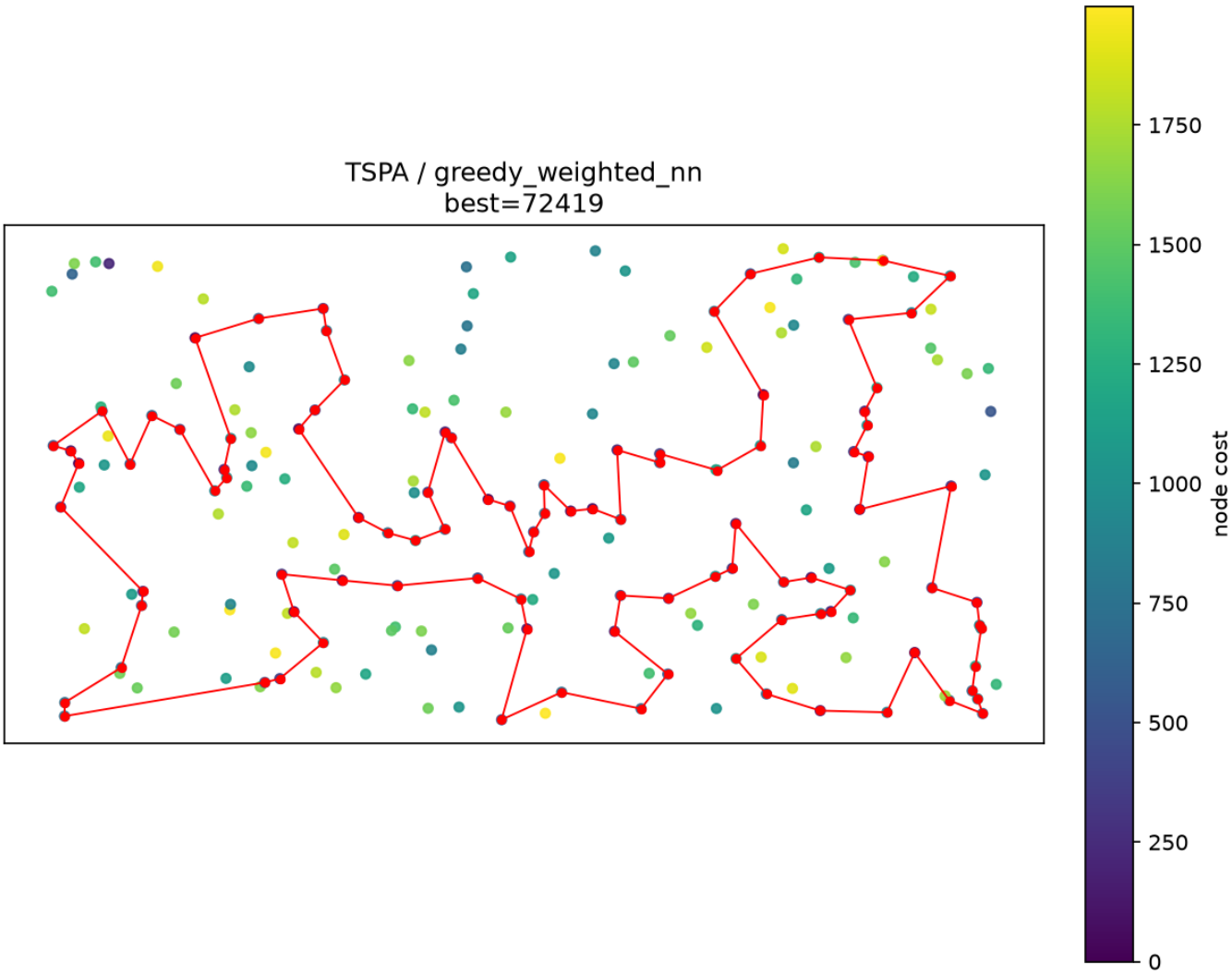
TSPA / greedy_2regret_nn — best=110950



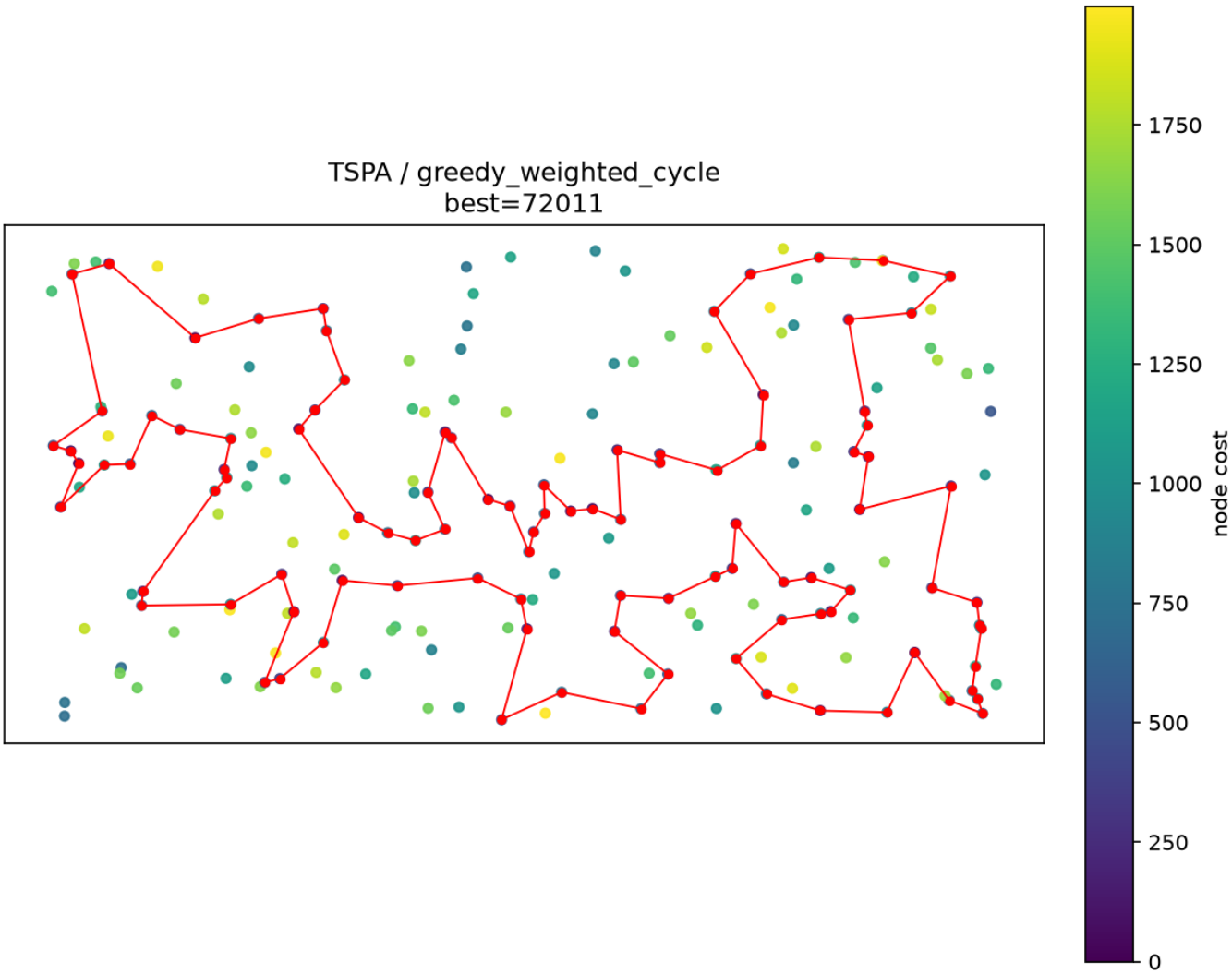
TSPA / greedy_2regret_cycle — best=112806



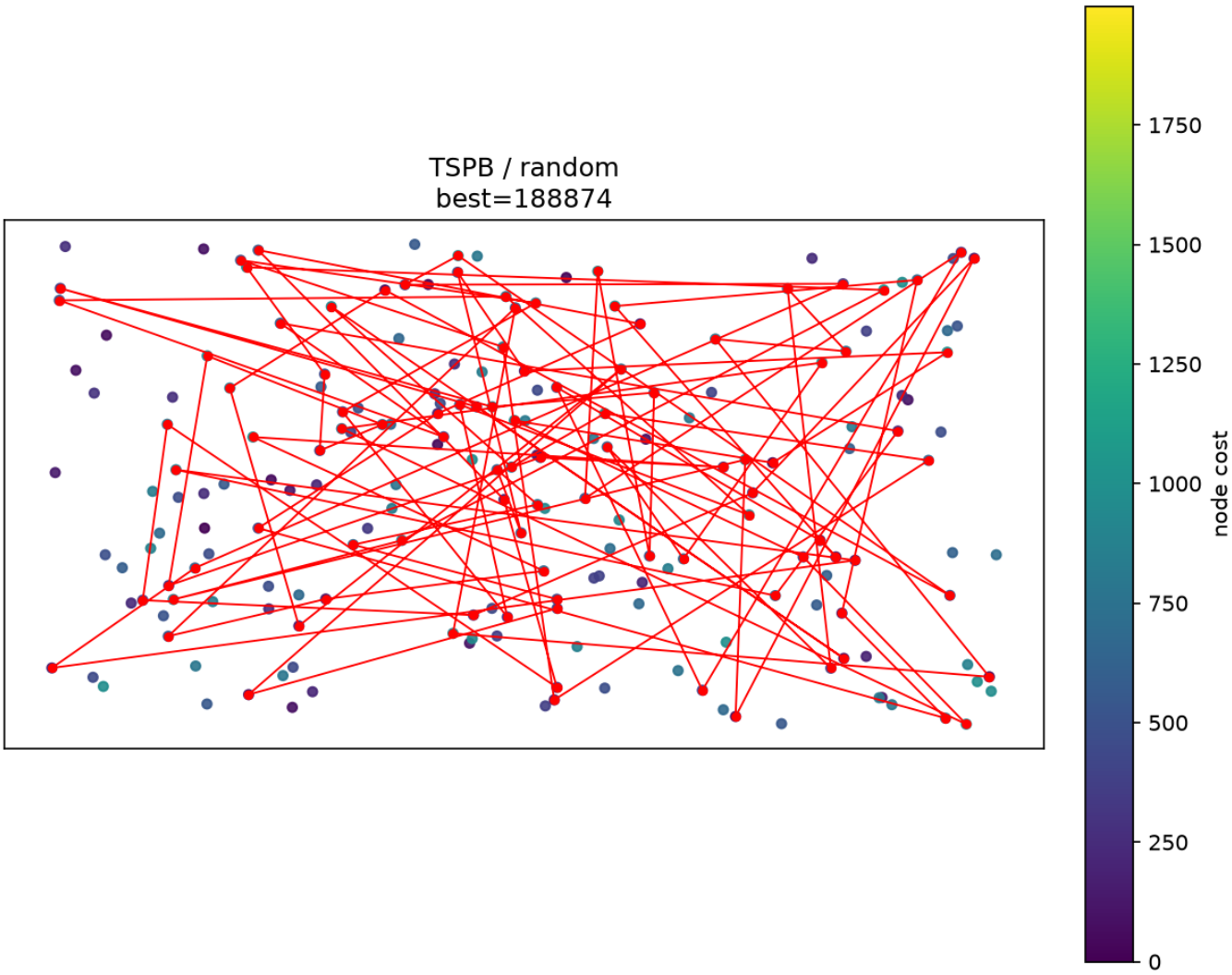
TSPA / greedy_weighted_nn — best=72419



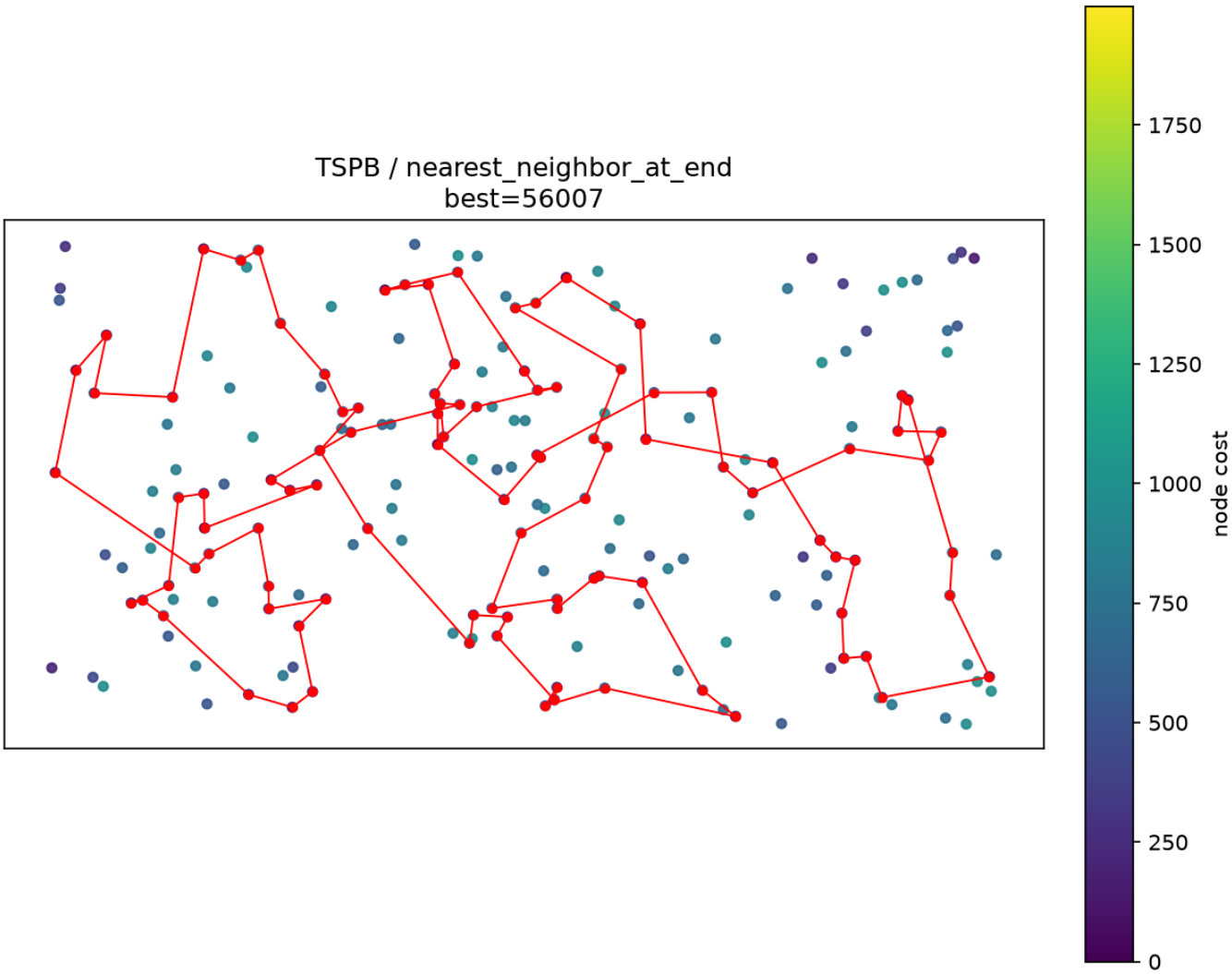
TSPA / greedy_weighted_cycle — best=72011



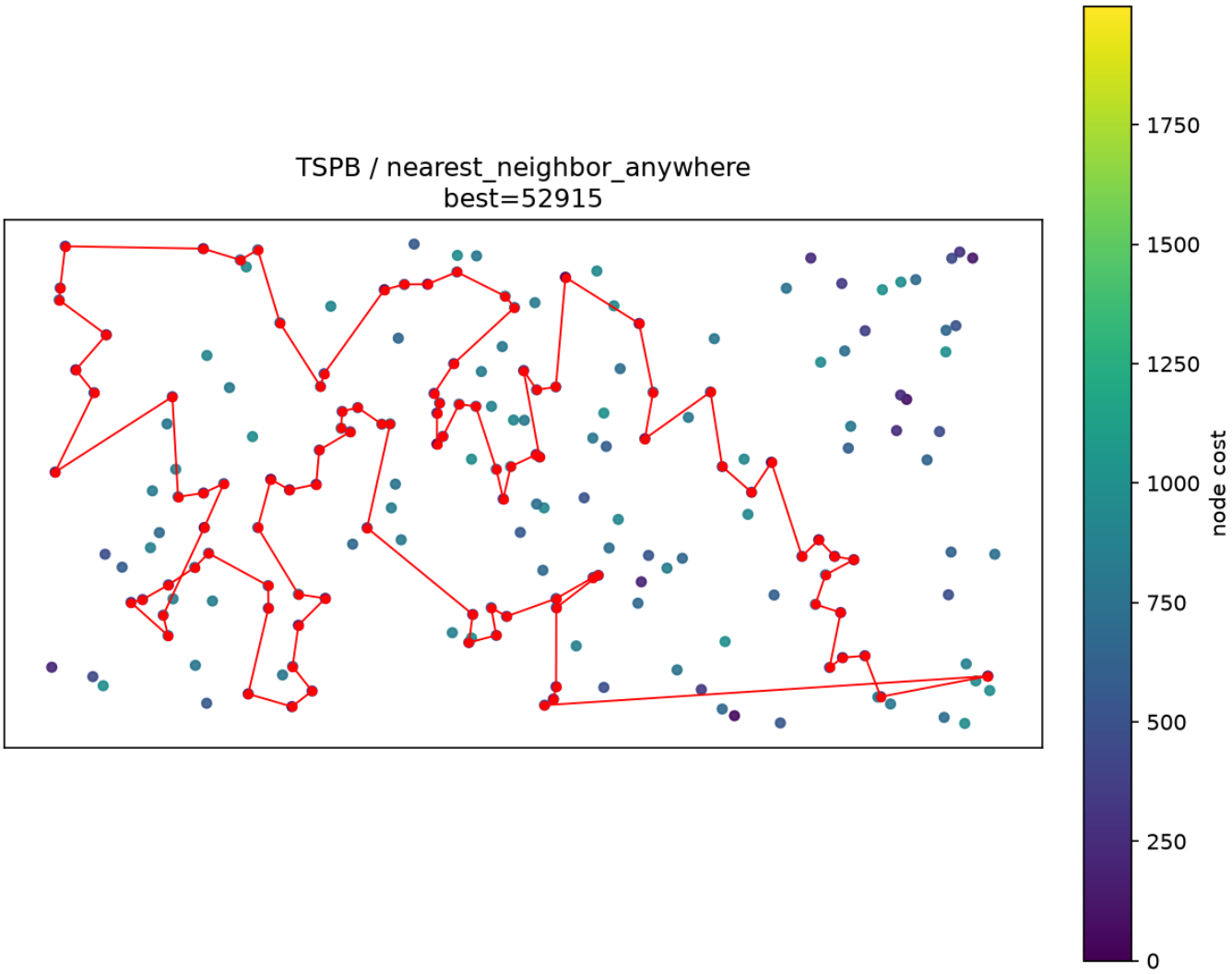
TSPB / random — best=188874



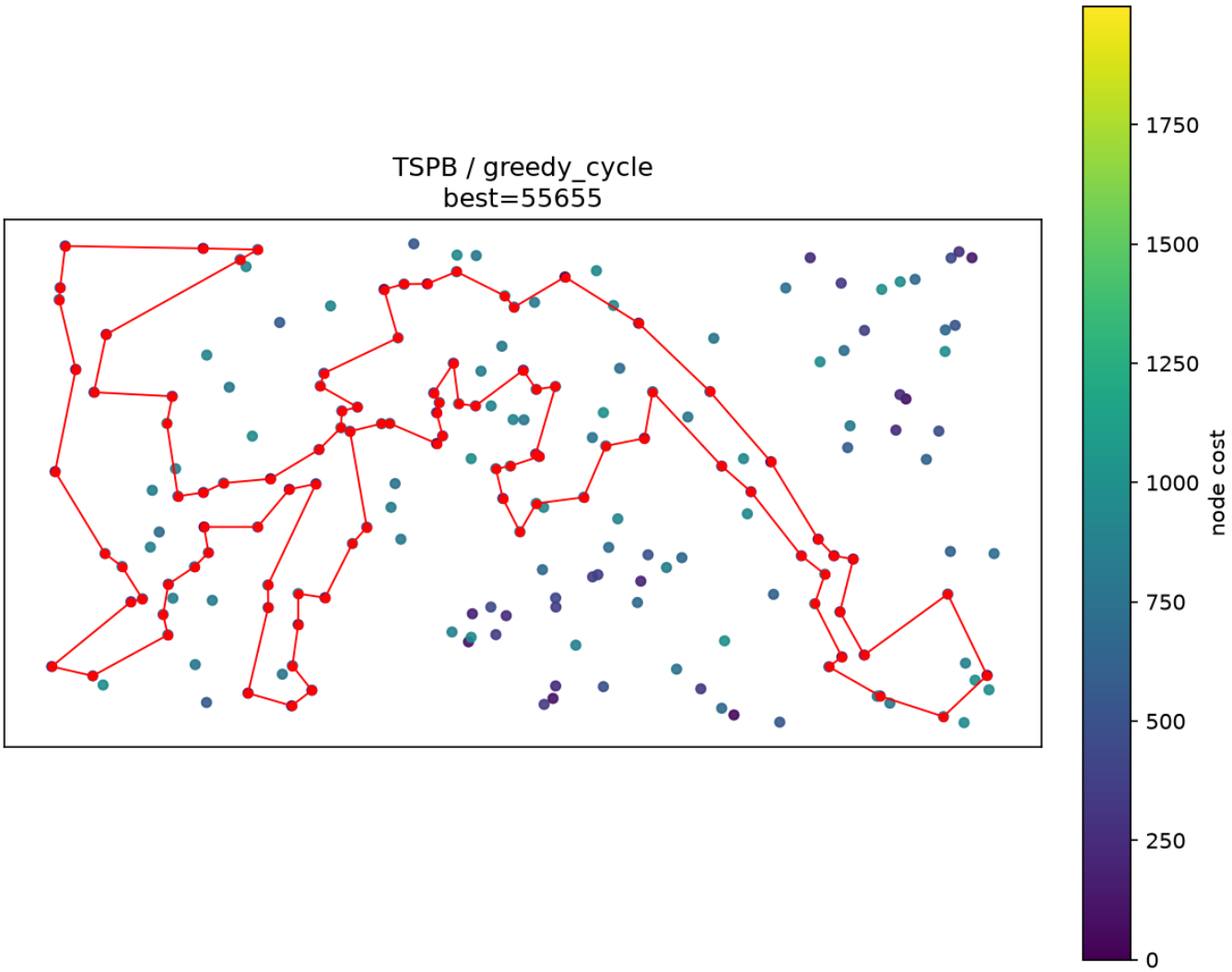
TSPB / nearest_neighbor_at_end — best=56007



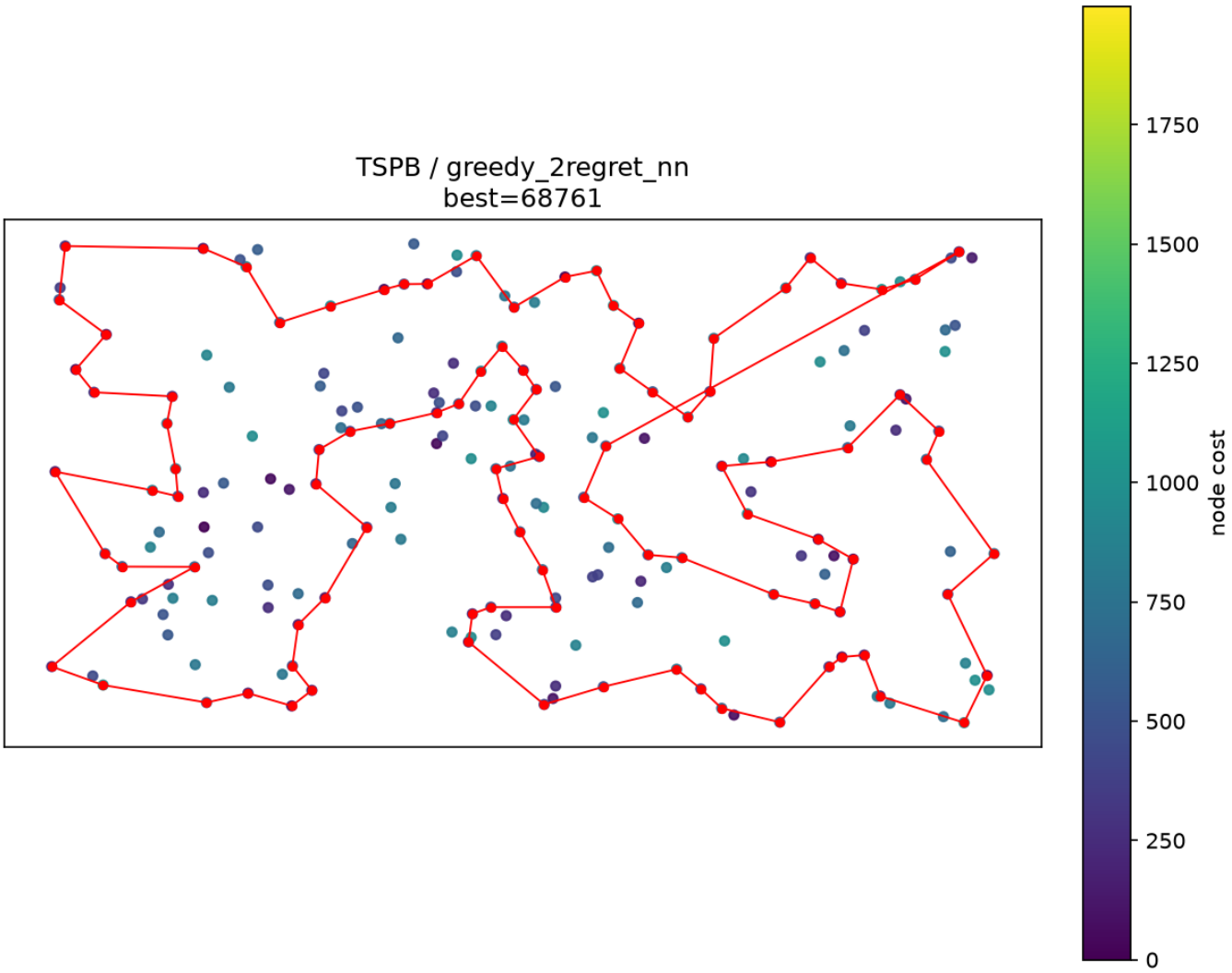
TSPB / nearest_neighbor_anywhere — best=52915



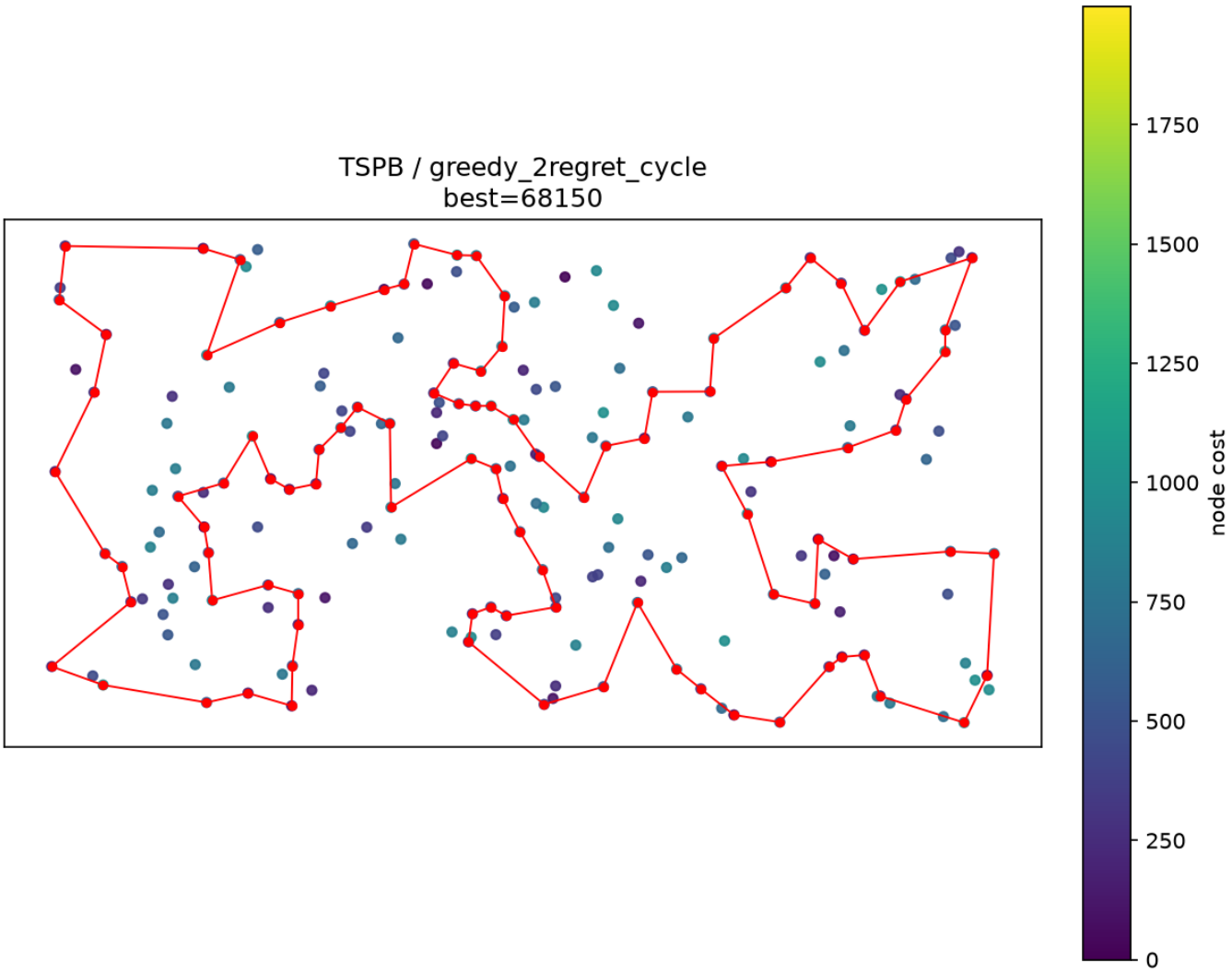
TSPB / greedy_cycle — best=55655



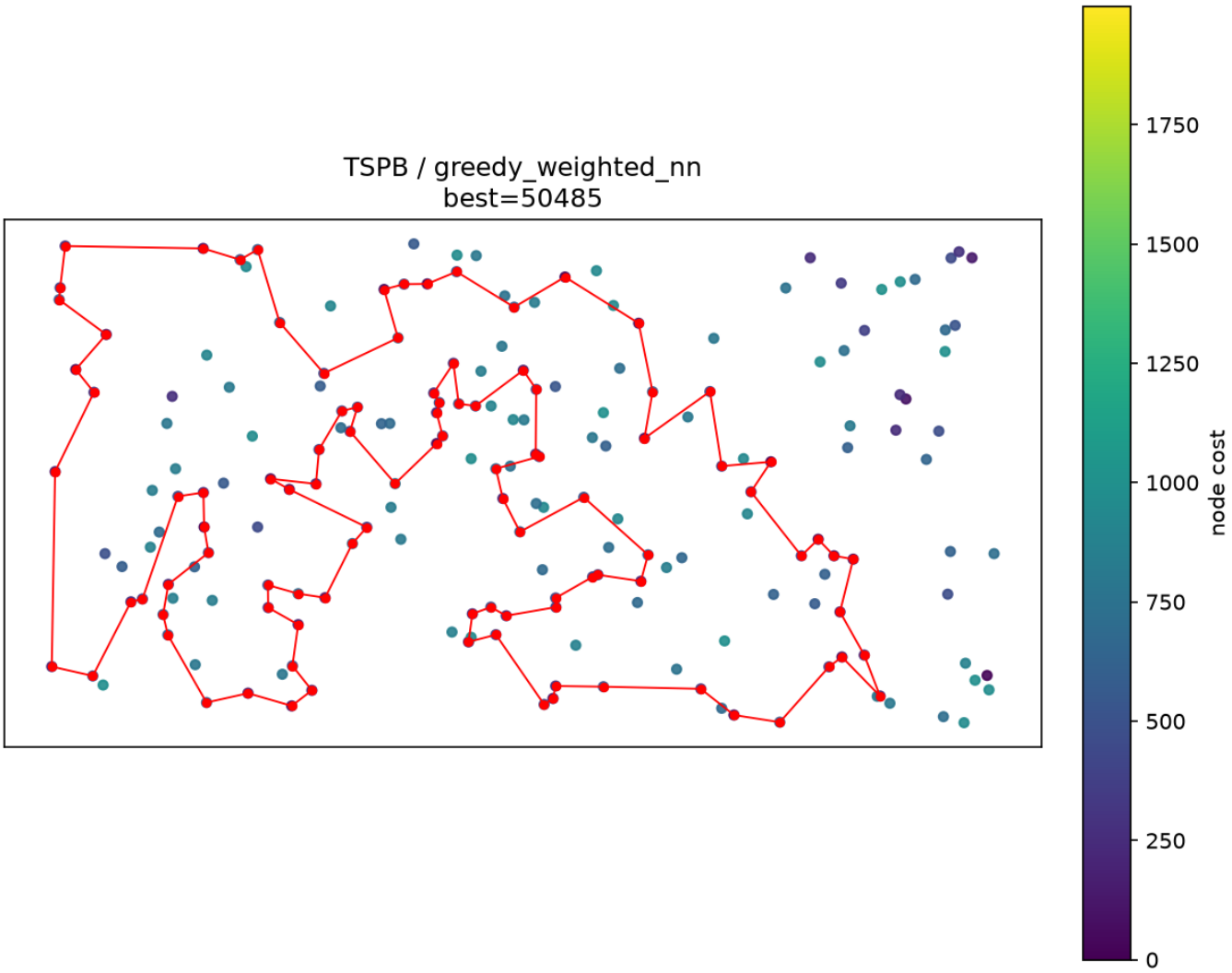
TSPB / greedy_2regret_nn — best=68761



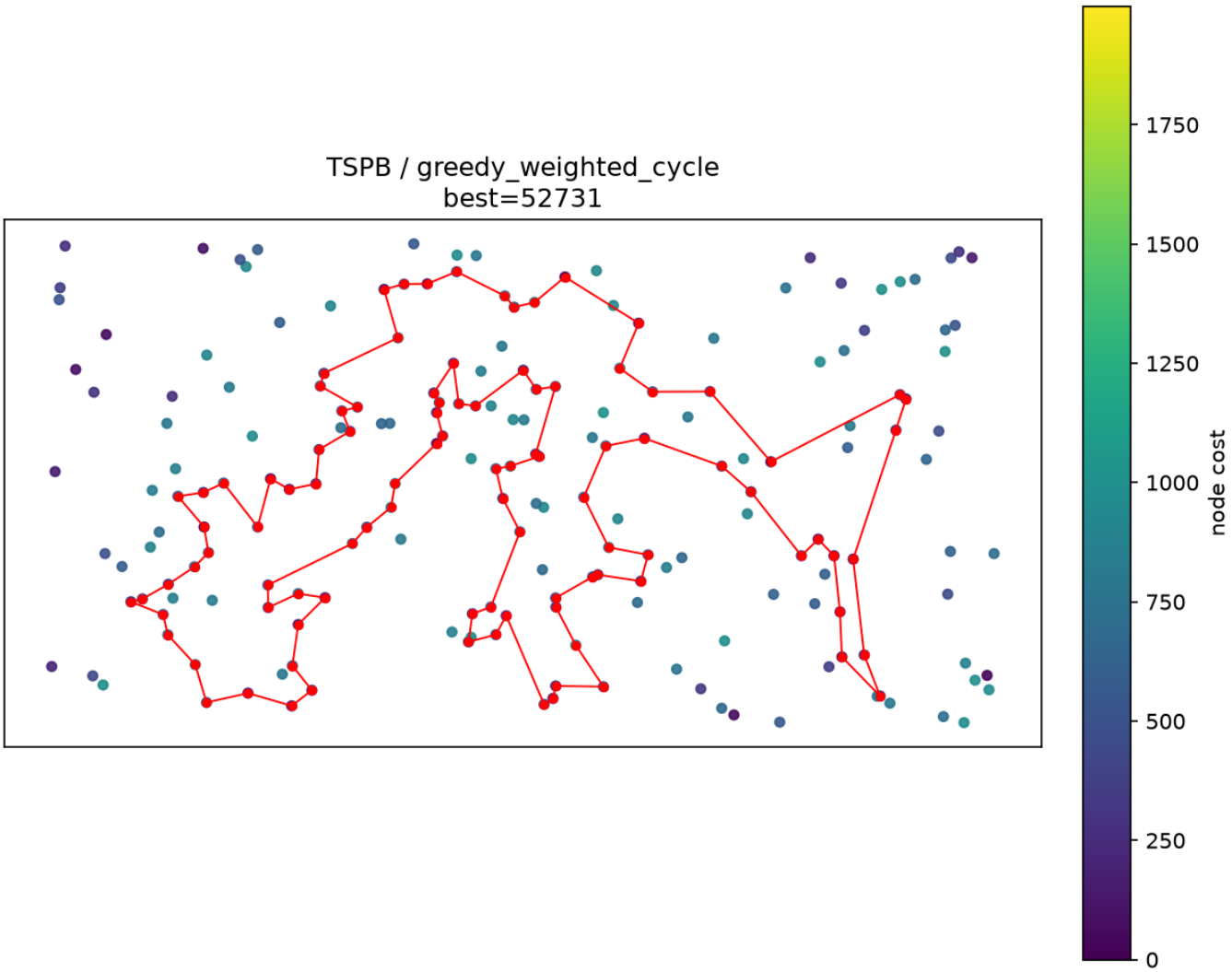
TSPB / greedy_2regret_cycle — best=68150



TSPB / greedy_weighted_nn — best=50485



TSPB / greedy_weighted_cycle — best=52731



4. Best solutions as node index lists (0-based)

TSPA

random:

```
[12, 18, 10, 187, 168, 160, 183, 171, 99, 80, 102, 144, 172, 88, 22, 135, 4, 92, 188, 59, 62, 86, 89,
151, 50, 35, 46, 190, 123, 111, 129, 198, 152, 25, 15, 98, 74, 126, 79, 84, 31, 191, 43, 20, 23, 28,
87, 109, 186, 78, 136, 29, 145, 56, 139, 162, 130, 9, 179, 189, 27, 195, 34, 142, 3, 181, 70, 149, 39,
150, 45, 167, 108, 7, 133, 105, 124, 184, 19, 52, 51, 65, 76, 82, 132, 166, 196, 42, 5, 21, 153, 66,
36, 97, 119, 161, 180, 158, 67, 17]
```

nearest_neighbor_at_end:

```
[54, 9, 33, 36, 177, 67, 144, 53, 135, 58, 134, 176, 174, 106, 89, 107, 57, 182, 151, 93, 50, 3, 1,
164, 178, 29, 136, 186, 184, 47, 159, 130, 153, 78, 88, 193, 170, 146, 75, 64, 150, 42, 113, 43, 85,
142, 8, 131, 84, 158, 192, 181, 114, 141, 180, 132, 76, 80, 48, 31, 63, 149, 168, 97, 127, 187, 30,
123, 152, 108, 60, 116, 49, 11, 119, 160, 83, 90, 51, 16, 35, 87, 45, 140, 52, 179, 185, 28, 102, 156,
157, 71, 91, 38, 100, 4, 112, 139, 115, 5]
```

nearest_neighbor_anywhere:

```
[104, 129, 69, 127, 97, 168, 116, 149, 63, 11, 49, 160, 83, 123, 100, 152, 108, 60, 31, 48, 80, 91,
157, 71, 4, 76, 132, 180, 28, 102, 185, 35, 156, 51, 16, 67, 144, 140, 53, 179, 52, 177, 36, 33, 9,
54, 131, 174, 8, 164, 186, 136, 163, 29, 101, 178, 1, 169, 3, 50, 147, 47, 93, 106, 42, 89, 113, 130,
159, 146, 170, 153, 193, 78, 88, 107, 64, 57, 182, 151, 150, 176, 72, 134, 84, 58, 158, 192, 181, 114,
135, 73, 45, 87, 141, 187, 30, 189, 2, 171]
```

greedy_cycle:

```
[31, 116, 149, 168, 63, 11, 49, 60, 108, 152, 100, 90, 123, 83, 160, 97, 127, 69, 171, 2, 189, 30,
187, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 150, 151, 182, 57, 64, 107, 88,
78, 193, 153, 170, 146, 159, 130, 113, 89, 42, 106, 93, 47, 147, 50, 3, 169, 1, 178, 101, 29, 163,
136, 186, 164, 142, 8, 174, 131, 54, 9, 33, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 156, 35, 185,
102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48]
```

greedy_2regret_nn:

```
[104, 69, 120, 97, 168, 149, 11, 49, 108, 152, 100, 123, 148, 162, 132, 76, 4, 157, 18, 173, 16, 156,
28, 185, 67, 144, 179, 177, 36, 33, 143, 190, 139, 112, 5, 115, 175, 39, 154, 34, 109, 186, 136, 197,
163, 19, 178, 101, 155, 21, 147, 37, 47, 50, 93, 81, 43, 113, 117, 130, 159, 170, 75, 26, 78, 122,
107, 64, 57, 105, 182, 111, 138, 165, 151, 150, 176, 72, 65, 134, 84, 58, 158, 192, 181, 118, 114, 25,
121, 73, 45, 70, 96, 10, 183, 166, 32, 40, 30, 23]
```

greedy_2regret_cycle:

```
[189, 166, 183, 10, 70, 22, 25, 114, 118, 181, 121, 61, 41, 140, 53, 179, 177, 36, 99, 84, 134, 72,
65, 20, 192, 111, 182, 165, 151, 106, 89, 124, 57, 64, 107, 88, 78, 26, 75, 170, 159, 130, 117, 93,
50, 47, 37, 147, 6, 46, 86, 55, 17, 29, 19, 163, 197, 136, 195, 184, 164, 142, 85, 81, 174, 194, 9,
33, 143, 62, 34, 154, 39, 175, 115, 5, 112, 139, 18, 4, 196, 51, 190, 67, 185, 156, 28, 180, 191, 141,
30, 160, 80, 82, 11, 168, 188, 120, 125, 187]
```

greedy_weighted_nn:

```
[171, 2, 189, 187, 30, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 174, 150, 106,
113, 89, 42, 151, 138, 182, 57, 64, 107, 88, 78, 193, 153, 75, 170, 146, 159, 130, 47, 93, 50, 3, 169,
1, 59, 178, 101, 55, 29, 163, 136, 186, 164, 142, 8, 9, 54, 33, 131, 36, 177, 52, 179, 53, 140, 144,
67, 16, 51, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 100, 123, 83, 160, 152, 108, 49, 60,
116, 149, 63, 168, 97, 127, 69, 129, 104]
```

greedy_weighted_cycle:

```
[31, 60, 116, 149, 63, 168, 11, 49, 108, 152, 100, 123, 83, 160, 97, 127, 161, 30, 187, 171, 2, 189,
141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 174, 150, 106, 113, 89, 42, 151, 138,
182, 57, 64, 107, 88, 78, 193, 153, 75, 170, 146, 159, 130, 47, 93, 50, 3, 169, 1, 178, 101, 55, 29,
163, 136, 186, 164, 142, 8, 9, 54, 33, 131, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 35, 185, 102,
28, 180, 132, 76, 4, 71, 157, 91, 80, 48]
```

TSPB

random:

```
[194, 74, 135, 84, 12, 176, 23, 80, 109, 54, 193, 72, 48, 98, 122, 111, 33, 152, 181, 175, 79, 60, 15,
121, 118, 16, 198, 70, 180, 1, 90, 55, 150, 10, 50, 160, 13, 43, 106, 8, 167, 30, 32, 147, 166, 87,
143, 0, 173, 195, 20, 117, 146, 95, 185, 66, 44, 27, 94, 99, 86, 127, 129, 83, 169, 153, 93, 97, 172,
57, 42, 151, 179, 138, 29, 59, 192, 105, 171, 159, 5, 49, 199, 7, 130, 38, 189, 123, 6, 3, 162, 161,
25, 40, 36, 156, 107, 110, 77, 18]
```

nearest_neighbor_at_end:

```
[32, 89, 57, 76, 54, 127, 129, 90, 153, 20, 14, 2, 110, 118, 56, 114, 196, 10, 148, 143, 78, 192, 162,
104, 185, 184, 3, 151, 69, 172, 133, 120, 96, 121, 198, 48, 72, 46, 86, 117, 160, 136, 8, 21, 154,
140, 47, 170, 125, 177, 6, 132, 29, 197, 175, 134, 45, 60, 40, 71, 155, 97, 115, 180, 65, 28, 149,
145, 158, 116, 147, 42, 50, 130, 30, 183, 38, 64, 85, 59, 167, 51, 87, 179, 82, 88, 171, 99, 67, 24,
34, 181, 152, 73, 98, 27, 135, 75, 79, 156]
```

nearest_neighbor_anywhere:

```
[110, 2, 14, 20, 74, 153, 182, 52, 90, 129, 127, 80, 54, 192, 162, 104, 76, 185, 57, 89, 86, 46, 72,
3, 184, 173, 151, 122, 117, 8, 160, 69, 172, 136, 133, 120, 156, 43, 48, 96, 198, 121, 130, 191, 50,
42, 147, 116, 37, 146, 13, 149, 28, 145, 65, 158, 177, 125, 26, 170, 197, 36, 132, 29, 6, 180, 115,
155, 71, 175, 134, 45, 157, 60, 40, 141, 97, 154, 140, 47, 38, 21, 176, 30, 183, 189, 168, 64, 59, 85,
51, 73, 167, 152, 34, 24, 181, 179, 87, 82]
```

greedy_cycle:

```
[174, 36, 197, 6, 180, 115, 170, 97, 140, 47, 155, 71, 175, 134, 45, 157, 60, 141, 40, 33, 64, 21,
189, 168, 69, 160, 172, 136, 133, 120, 8, 117, 72, 46, 86, 184, 3, 173, 122, 151, 98, 111, 27, 135,
76, 185, 162, 192, 80, 52, 182, 20, 74, 2, 93, 110, 118, 14, 153, 90, 129, 127, 54, 104, 57, 89, 156,
43, 48, 96, 198, 121, 63, 130, 191, 183, 30, 176, 38, 154, 26, 125, 177, 138, 158, 145, 149, 147, 42,
116, 37, 146, 13, 28, 65, 178, 103, 29, 132, 1]
```

greedy_2regret_nn:

```
[94, 169, 199, 70, 190, 194, 77, 104, 144, 185, 79, 57, 83, 44, 89, 156, 164, 96, 198, 121, 195, 50,
7, 116, 37, 13, 149, 28, 145, 158, 138, 55, 177, 53, 65, 178, 103, 180, 132, 1, 108, 62, 175, 134, 45,
157, 60, 40, 64, 47, 38, 21, 168, 172, 8, 92, 166, 72, 46, 109, 3, 122, 151, 98, 25, 181, 73, 59, 85,
82, 88, 163, 99, 39, 17, 74, 20, 14, 2, 23, 110, 118, 131, 143, 148, 196, 78, 54, 162, 106, 127, 90,
153, 182, 150, 84, 66, 35, 27, 135]
```

greedy_2regret_cycle:

```
[73, 167, 181, 25, 98, 151, 122, 128, 4, 168, 183, 176, 38, 47, 140, 154, 161, 26, 177, 170, 115, 113,
155, 141, 60, 157, 134, 175, 62, 108, 1, 132, 103, 178, 65, 145, 149, 13, 37, 116, 147, 123, 50, 195,
121, 198, 58, 16, 164, 43, 166, 92, 120, 133, 8, 117, 95, 109, 3, 27, 135, 76, 185, 104, 77, 194, 190,
70, 187, 165, 105, 139, 193, 114, 10, 78, 54, 162, 106, 150, 182, 127, 90, 56, 131, 110, 23, 2, 14,
20, 74, 17, 171, 99, 163, 31, 88, 82, 85, 59]
```

greedy_weighted_nn:

```
[65, 145, 28, 149, 13, 146, 37, 116, 147, 42, 50, 130, 63, 121, 198, 96, 48, 156, 89, 57, 185, 76,
104, 162, 54, 192, 80, 127, 129, 90, 153, 14, 2, 20, 74, 17, 171, 99, 88, 179, 87, 82, 51, 85, 59, 73,
167, 181, 152, 34, 24, 67, 66, 27, 98, 151, 122, 3, 184, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69,
124, 21, 183, 30, 38, 47, 154, 140, 64, 33, 40, 141, 155, 71, 60, 157, 45, 134, 175, 62, 36, 197, 6,
115, 170, 125, 177, 29, 132, 174, 1]
```

greedy_weighted_cycle:

```
[91, 36, 197, 132, 29, 6, 180, 115, 170, 177, 125, 26, 97, 154, 140, 47, 38, 21, 30, 183, 191, 130,
63, 121, 198, 96, 48, 43, 156, 32, 89, 57, 79, 185, 104, 54, 196, 114, 10, 90, 14, 2, 20, 153, 129,
127, 80, 192, 162, 76, 135, 27, 11, 66, 67, 24, 34, 152, 181, 126, 88, 179, 87, 82, 167, 51, 85, 59,
73, 98, 151, 122, 173, 3, 184, 86, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 124, 4, 64, 33, 155,
71, 141, 40, 60, 157, 45, 134, 175, 62]
```

5. Solution checking

All 16 best tours were checked manually with the solution-checker spreadsheet (`solutions/lab2_*_ids.csv` pasted into `F3:F102`, total read from `K2`) and match the costs above.

```
TSPA/random: best_ok=True best_cost=237157
TSPA/nearest_neighbor_at_end: best_ok=True best_cost=84362
TSPA/nearest_neighbor_anywhere: best_ok=True best_cost=72328
TSPA/greedy_cycle: best_ok=True best_cost=72590
TSPA/greedy_2regret_nn: best_ok=True best_cost=110950
TSPA/greedy_2regret_cycle: best_ok=True best_cost=112806
TSPA/greedy_weighted_nn: best_ok=True best_cost=72419
TSPA/greedy_weighted_cycle: best_ok=True best_cost=72011
TSPB/random: best_ok=True best_cost=188874
TSPB/nearest_neighbor_at_end: best_ok=True best_cost=56007
TSPB/nearest_neighbor_anywhere: best_ok=True best_cost=52915
TSPB/greedy_cycle: best_ok=True best_cost=55655
TSPB/greedy_2regret_nn: best_ok=True best_cost=68761
TSPB/greedy_2regret_cycle: best_ok=True best_cost=68150
TSPB/greedy_weighted_nn: best_ok=True best_cost=50485
TSPB/greedy_weighted_cycle: best_ok=True best_cost=52731
```

No duplicates, no out-of-range indices, no length violations.

6. Source code

https://github.com/skalskidaniel/evolutionary-computation/blob/main/lab2.ipynb

7. Conclusions

1. **Regret alone fails here; regret + price wins.** Pure 2-regret loses even to the weakest Lab 1 greedy (TSPA 110950 vs 84362; TSPB 68150 vs 56007 on best), because it chases insertion urgency while ignoring absolute cost. Weighting regret against best change (1:1) turns the same construction into the overall winner on both instances.

2. **Weighted regret beats Lab 1 everywhere that matters.** Best tour: 72328 → 72011 (TSPA), 52915 → 50485 (TSPB, –4.6%). Best mean: 73793.4 → 73353.5 (TSPA), 59480.3/59397.3 → 55698.9 (TSPB). The Lab 1 ranking is otherwise preserved: NN-anywhere still beats NN-end on best in both instances.
3. **Base matters by instance.** TSPA favors the cycle base (weighted-cycle takes best and mean); TSPB favors the path base for best/mean, with the cycle keeping the better worst case. Never assume one insertion topology dominates — the selection component breaks that intuition.
4. **Weights are robust, not touchy.** Across `(1.0,1.0)`, `(1.0,0.5)`, `(0.5,1.0)` the best tour moves by ~2% on TSPA; on TSPB the regret-heavy path config even takes the best mean (54710.1). Equal weights are a safe default; tuning buys single percent, not tens.
5. **Cost term stays load-bearing.** All deltas include `+C[c]`; the pure-regret failure mode is exactly what happens when urgency outweighs price. Visualizations confirm winners thread through low-cost regions, not just short edges.
6. **Next steps:** local search (2-opt) on top of weighted starts; regret variants with more than two positions (k-regret); adaptive weights during construction.